Übungsblatt 6: Das Steuerwerk

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Einen Decoderentwurf korrigieren (12 Punkte)

Single-Cycle-Kursmodell mit addi-Erweiterung. ALUSrc: 0 = RD2, 1 = Immediate. MemtoReg: 0 = ALU, 1 = Speicher. X ist nur bei gesperrter Registerübernahme unerheblich.

1. Prüfen Sie jede Zeile. Korrigieren Sie falsche Bits; kennzeichnen Sie auch eine vollständig richtige Zeile. (5 P)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
or	0	0	1	0	0	0
addi	1	1	1	0	0	0
sw	1	X	1	0	1	0
lw	1	1	1	0	0	0
beq	1	X	0	0	0	1

2. Ordnen Sie jeder korrigierten Zeile ALUOp zu: 00 = ADD, 01 = SUB-Vergleich, 10 = Register-Funktionsfelder. Warum gehört addi zu 00? (3 P)
3. Dürfen beim korrigierten sw sowohl MemtoReg = 0 als auch 1 verwendet werden? Begründen Sie. (2 P)
4. Wie viele benannte Hauptsignale und Bits enthalten diese sechs Signale zusammen mit ALUOp? Ist PCSrc ein weiterer unabhängiger Hauptausgang? (2 P)

Musterlösung

1. or korrekt. addi: MemtoReg muss 0 sein. sw: RegWrite muss 0 sein. lw: MemRead muss 1 sein. beq: ALUSrc muss 0 sein. Je richtige Zeilenentscheidung 1 P.
2. Kategorien in Tabellenreihenfolge: 10, 00, 00, 00, 01. Je 0,5 P. addi benötigt unabhängig von den Immediate-Bits immer Addition (0,5 P).
3. Ja. RegWrite = 0 verhindert jede Registerübernahme; die Store-Daten kommen separat aus RD2. Urteil und Datenweg je 1 P.
4. Sieben Signale, acht Bits wegen des zweibittigen ALUOp (zusammen 1 P). PCSrc wird aus Branch und Zero abgeleitet (1 P).

Aufgabe 2: Rechenkategorie und Detailkommando (10 Punkte)

Unabhängige Fälle mit a2 = 25, a3 = 12. Gültige Register-Funktionsfelder: AND = (funct3 111, funct7 0000000), SUB = (000, 0100000). addi hat Opcode 0010011 und funct3 000.

Interne ALUControl-Codes: AND 0000, OR 0001, ADD 0010, SUB 0110.

1. Bestimmen Sie ALUOp, ALUControl, Quelle B und Ergebnis für alle Zeilen. (8 P)

```
and  a1, a2, a3
sub  a1, a3, a2
addi a1, a2, -13
addi a1, a2, 1536
```

2. Warum verlangt das gesetzte Instruktionsbit 30 der letzten Zeile keine Subtraktion? (1 P)
3. Ein Registermuster hat funct3 = 110 und funct7 = 0100000. Darf unser Decoder daraus OR ableiten, obwohl nur das Paar (110, 0000000) als OR definiert wurde? (1 P)

Musterlösung

Zeile	ALUOp	ALUControl	Quelle B	Ergebnis a1
AND	10	0000	a3 = 12	8
SUB	10	0110	a2 = 25	−13
ADDI −13	00	0010	Immediate −13	12
ADDI 1536	00	0010	Immediate 1536	1561

1. Je Eintrag 0,5 P.
2. Bit 30 ist bei addi ein Immediate-Bit. Kategorie 00 erzwingt ADD; das Wort ist kein R-Format. 1 P.
3. Nein. Nur die vollständige gültige Kombination darf als unterstützte OR-Instruktion akzeptiert werden; eine einzelne Feldübereinstimmung genügt nicht. 1 P.

Aufgabe 3: Neue Boolesche Steuerbedingungen (10 Punkte)

R, L, S, B, I sind gegenseitig ausschließende Erkennungssignale für Registeroperationen, lw, sw, beq und addi. ALUOp verwendet 10 für R, 01 für B, sonst 00.

1. Leiten Sie die beiden einzelnen ALUOp-Bits sowie MemRead und MemWrite als Boolesche Funktionen her. (4 P)
2. Prüfen Sie die Gleichwertigkeit der beiden Rückschreibauswahlen für alle fünf Gruppen: $W_1 = L \vee B$ und $W_2 = L \vee S \vee B$. Warum bleiben beide zulässig? (3 P)
3. Warum wäre $W_3 = L \vee I$ als MemtoReg-Funktion falsch? (1 P)
4. Eine andere Funktion ist für $(A, B) = (0, 0)$ frei, bei (0,1) null und bei (1,0), (1,1) eins. Geben Sie zwei unterschiedliche zulässige Formeln an. (2 P)

Musterlösung

1. ALUOp$_1 = R$, ALUOp$_0 = B$, MemRead $= L$, MemWrite $= S$. Je 1 P.

Gruppe	R	L	S	B	I
W_1	0	1	0	1	0
W_2	0	1	1	1	0

2. Tabellen je 1 P, Begründung 1 P: Der einzige Unterschied liegt bei sw, dessen RegWrite null ist. Auch bei beq darf die Auswahl eins sein.
3. Bei addi würde Speicher statt ALU-Ergebnis zurückgeschrieben; dort ist RegWrite eins. 1 P.
4. Beispielsweise $f = A$ und $g = A \vee \overline{B}$. Beide erfüllen die drei festgelegten Zeilen, unterscheiden sich aber bei (0,0). Je 1 P. X bezeichnet hier Entwurfsfreiheit, keinen elektrischen dritten Logikpegel.

Aufgabe 4: Fehler mit genau bestimmten Nebenwirkungen (10 Punkte)

Gültige, beschreibbare, natürlich ausgerichtete Adressen; Little Endian. Jeder Fall startet unabhängig.

1. `or t0, t1, t2` mit t1 = `0x9000` und t2 = `0x10`. Nur MemWrite ist irrtümlich eins; das betroffene Speicherwort war −1. Bestimmen Sie Speicheradresse, Store-Daten, t0 und neue Bytes. (4 P)
2. `sw a1, -8(s2)` mit s2 = `0x900C`, a1 = 128. Fehler: RegWrite = 1 und MemtoReg = 0. Der Registeradressport erhält Bits 11:7 des Befehlsworts. Bestimmen Sie regulären Store und zusätzlich beschriebenes Register samt Wert. (4 P)
3. Ein anderer Store nutzt Offset −32 bei denselben fehlerhaften Freigaben. Was ändert sich am zusätzlichen Registereffekt, und weshalb? (2 P)

Musterlösung

1. OR-Ergebnis und Adresse `0x9010`; Speicher-Schreibdaten aus RD2 = 16; t0 = `0x9010`; Bytes `10 00 00 00`. Je 1 P.
2. Regulärer Store: Adresse `0x9004`, Daten 128 (zusammen 2 P). Die unteren fünf Immediate-Bits von −8 sind `11000` = 24: x24 erhält zusätzlich die ALU-Adresse `0x9004` (zusammen 2 P).
3. −32 hat untere fünf Bits `00000`, also Ziel x0. Der falsche Registerschreibversuch wird verworfen; es gibt keinen zusätzlichen sichtbaren Registereffekt. Ziel und Erklärung je 1 P. Der eigentliche Store erfolgt weiterhin an `0x8FEC`.

Aufgabe 5: Null ist noch keine Sprungfreigabe (8 Punkte)

Befehlsadresse `0x5A0`, fertiger Branch-Byteoffset −32. Drei unabhängige Fälle:

Fall	Befehl	Operanden
A	beq	11, 11
B	beq	11, −11
C	addi a0, x0, 0	0, Immediate 0

1. Bestimmen Sie jeweils Branch, Zero, PCSrc und nächsten PC. (4 P)
2. Nennen Sie ALUOp und ALUControl für die drei Fälle. Kurscodes aus Aufgabe 2 gelten. (2 P)
3. Warum ist Branch in B trotzdem eins? Warum ist PCSrc in C trotz Zero eins gleich null? (2 P)

Musterlösung

Fall	Branch	Zero	PCSrc	nächster PC
A	1	1	1	`0x580`
B	1	0	0	`0x5A4`
C	0	1	0	`0x5A4`

1. Je Eintrag 1/3 P.
2. A und B: ALUOp 01, ALUControl 0110; C: ALUOp 00, ALUControl 0010. Je korrektem Code 1/3 P.
3. Branch kennzeichnet die Befehlsfamilie (1 P); PCSrc verlangt Branch und Zero gleichzeitig, und addi ist kein Branch (1 P).

Aufgabe 6: Eine Bitprüfung in Ripes

1. Öffnen Sie `beispiele/lab_06_bitpruefung.s` im RV32-Single-Cycle-Modell ohne C. Protokollieren Sie Version und Modell.
2. Beginnen Sie bei Beobachtung. Notieren Sie vor jeder Clock Rechenoperation, ALU-Quelle, Rückschreibquelle und Schreibfreigaben; danach Register, Speicherwort und PC.
3. Begründen Sie vor dem Lauf, welcher Pfad der Branch nehmen muss. Prüfen Sie t0 bis t5 und das zweite Pufferwort bei inspect.
4. Verändern Sie in einem separaten Lauf das zweite Speicherwort zwischen Store und Load auf null. Welcher Pfad und t5-Wert entstehen jetzt?
5. Erklären Sie am Store, weshalb ein konkreter Rückschreib-MUX-Wert mit X in Ihrer Papiertabelle vereinbar ist.

Musterlösung

Regulär: t0 = −13, t1 = 8, t2 = −5, zweites Wort = −5, t3 = −5, t4 = 8. Der Branch wird genommen und t5 = 12. Bitweise gilt $(A \vee B) \wedge B = B$; hier liefert die Maske daher genau t1.

Nach der gezielten Speicheränderung liest t3 null und t4 wird null. Der Branch ist nicht genommen, t5 = −1. Die Software setzt inspect über einen separaten beq-Jump fort und erreicht Pass nicht.

Beim Store ist RegWrite null. Der gewählte Rückschreibwert wird nicht übernommen; die Speicher-Schreibdaten kommen aus RD2. Die Tabelle beschreibt Sollwerte, keine bereits erfolgten Ripes-Beobachtungen.